Class 23: Process Models and Agile Foundations

Session 23 of 36 – MLOps Block

Data-driven Systems Engineering

Agenda

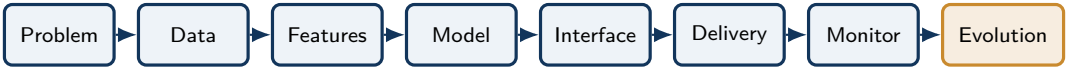
- Why process models exist and what they control
- Process descriptions and the four core activities
- Plan-driven, incremental, and hybrid approaches
- Waterfall versus agile in software and ML work
- Agile history, manifesto, principles, and drawbacks
- Process selection for ML-enabled systems

Process models are decision tools

Learning goals

- Compare plan-driven, incremental, and hybrid approaches.
- Understand why ML projects often need hybrid processes.
- Match process choice to uncertainty, regulation, and team maturity.
- Explain how process decisions affect documentation, testing, and release risk.

Course Map and Running Cases



Today in the course

Focus: Choosing a development process that fits uncertainty and risk.
Bridge: This class bridges from requirements into delivery strategy by asking how teams should organize work across exploration, validation, and release.

Fraud case

In the fraud case, agile iteration helps with thresholds and features, while controlled release gates protect the production path.

Pasteurization case

In the pasteurization case, exploratory modeling may be agile, but operational deployment benefits from stronger validation and change control.

Course thread: The course thread is that process choice shapes how every later activity gets coordinated.

Why software teams need process models

- A process model makes coordination explicit: who decides, who reviews, and what evidence is required.
- It reduces confusion around sequencing, approvals, and handoffs.
- It creates a shared language for planning, validation, and improvement.
- In ML systems, it also helps separate exploratory work from production commitments.

Key idea

Process is not bureaucracy by definition. A good process reduces friction by clarifying how work moves from idea to dependable outcome.

Process descriptions

- Activities: for example data modeling, UI design, service integration.
- Products: documents, code, models, dashboards, tests.
- Roles: analyst, developer, data scientist, reviewer, operator.
- Pre- and post-conditions: what must be true before and after an activity.

Recap: the four core process activities

- Class 20 already introduced specification, development, validation, and evolution as the backbone of any software process.
- This class does not repeat that material; it asks how those four activities get organized in time and by whom.

ML-specific nuance, carried forward

Evolution includes code change, data change, feature change, threshold change, and sometimes retraining under new operating conditions.

Waterfall, agile, and hybrid in one view

Model	Strength	Main risk
Waterfall	Strong control and documentation	Adapts poorly to late discoveries
Agile	Fast feedback and iteration	Can drift without disciplined scope control
Hybrid	Exploration plus governance	Needs explicit handoff rules

Plan-driven processes

- Activities are planned in advance and progress is measured against the plan.
- Works well when scope is stable, interfaces are fixed, and coordination cost is high.
- Documentation and formal review are first-class artifacts, not afterthoughts.
- Best fit often appears in safety-critical, regulated, or externally audited systems.

Example

Healthcare decision support, industrial control, or finance platforms may need strong traceability before release.

Incremental and agile processes

- Work is delivered in usable increments so learning can happen early.
- Specification, design, and implementation become interleaved.
- Customer feedback changes priorities continuously.
- Small releases reduce the cost of being wrong, but only if the team keeps a strong definition of done.

Waterfall versus agile in this course

Waterfall strengths

- Stable scope
- Formal sign-off
- Easier external audit

Agile strengths

- Fast feedback
- Iterative learning
- Better fit for uncertain data work

Incremental delivery reduces downtime risk

- In a monolithic release, many changes arrive at once and rollback becomes expensive.
- In incremental delivery, small changes can be isolated, tested, and reverted more safely.
- This is one reason CI/CD and agile practices complement each other.

Example

An e-commerce platform can deploy login first, then cart, then payment. A fraud API can deploy logging, then scoring, then alert routing instead of changing everything at once.

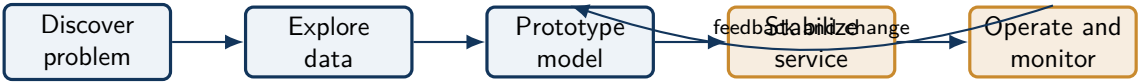
Why ML work often needs hybrid delivery

- Early phases are exploratory: data quality, labels, and metrics are uncertain.
- Later phases need discipline: deployment, interfaces, testing, and compliance.
- Regulated contexts cannot rely on verbal agreements or informal notebooks.

Example

An industrial monitoring system may use agile prototyping for the predictor, then a more controlled process for deployment in the plant network.

A hybrid lifecycle for data-driven systems



- The point of a hybrid process is matching control to uncertainty.

Useful classroom comparison

Fraud API project

- Agile for feature and threshold iteration
- Structured release gates
- Monitoring after deployment

Hospital ML support tool

- Stronger requirement control
- Formal validation and logging
- Slower but safer release cadence

Why agile emerged

- Agile methods grew partly from dissatisfaction with heavyweight software methods in the 1980s and 1990s.
- Teams wanted faster delivery, lower overhead, and better response to changing requirements.
- The emphasis shifted from exhaustive up-front design toward working software and rapid learning.

Important caution

Agile was never meant to mean “no design” or “no documentation.” It means documentation and planning should earn their cost.

Agile manifesto and principles

- Individuals and interactions over processes and tools.
- Working software over comprehensive documentation.
- Customer collaboration over contract negotiation.
- Responding to change over following a plan.

Important nuance

The right side still matters. In ML projects, documentation, governance, and change control return quickly once deployment begins.

Agile principles adapted to ML teams

- Customer involvement: stakeholders help define value and review increments.
- Incremental delivery: baselines, APIs, dashboards, and monitoring can each be delivered in stages.
- People over process: expertise matters when requirements and data are still uncertain.
- Embrace change: feedback from errors, labels, or operations is expected.
- Maintain simplicity: avoid unnecessary features, pipelines, and tools.

Drawbacks and failure modes

- Customers may disengage from the constant feedback cycle.
- Multiple stakeholders may prioritize conflicting changes.
- Teams can confuse speed with lack of discipline.
- Without clear definitions of done, iteration turns into churn.

How to choose a process model

Question	Process implication
Are requirements stable?	More stability supports plan-driven structure.
Is data quality uncertain?	More uncertainty favors iterative exploration.
Is the system regulated?	More regulation requires stronger documentation and approval.
Are releases frequent?	Frequent change favors incremental delivery and automation.
Will the model evolve in production?	Evolution demands monitoring and change governance.

What students should defend in presentations

- Why they chose their process.
- Where requirements may still change.
- Which artifacts create team alignment.
- When they move from experimentation to controlled delivery.

From This Class to the Next

What stays with us

- No process model is universally best; the fit depends on uncertainty, feedback speed, and governance needs.
- ML projects often need iterative discovery without losing engineering control.
- Process choice affects documentation, coordination, testing, and release rhythm.

Project use

Project teams should now be able to justify whether their work needs more exploratory loops, more structure, or a hybrid approach.

Next connection

Class 25 narrows that process discussion into practical agile tools such as backlogs, stories, boards, ceremonies, and feedback loops.

Course thread: After selecting a development posture, we translate it into everyday coordination practices the team can actually run.

Papers and materials

Papers and materials

Agile Manifesto; Sommerville, Software Engineering; Martin Fowler on the new methodology; The Scrum Guide; Production AI lifecycle discussion in ACM Queue.